

Git & Bitbucket

Java Backend Interview — Short Notes

1. Git Fundamentals

1.1 How Git Works Internally

- **Blob**: stores file content (hashed with SHA-1/SHA-256). Same content = same blob (deduplication).
- **Tree**: stores directory listing — maps filenames to blob/tree hashes.
- **Commit**: snapshot pointer — tree hash + parent commit(s) + author + message + timestamp.
- **Tag**: named pointer to a commit. Lightweight (pointer) or Annotated (full object with message).
- **Branch**: movable pointer to a commit. HEAD = pointer to current branch/commit.
- Git is a **DAG (Directed Acyclic Graph)** of commits. Merge = two parents. Rebase = new commits.

1.2 Three Areas

Area	Description	Commands to Move	Notes
Working Directory	Your local files. Untracked or modified tracked files.	git add → staging. git restore → undo changes.	Not yet in Git. 'git status' shows state.
Staging Area (Index)	Prepared snapshot for next commit. Only staged changes go into commit.	git commit → repo. git restore --staged → unstage.	Allows partial commits — stage only some files/hunks.
Repository (.git)	All commits, branches, history. The actual Git database.	git checkout/reset/revert → working dir.	Local repo. git push syncs to remote.

2. Essential Git Commands

2.1 Daily Workflow

Command	What It Does	Example
git init	Initialize new repo.	git init my-project
git clone	Clone remote repo locally.	git clone https://github.com/org/repo.git
git status	Show working dir and staging area state.	git status -s # short format
git add	Stage files/changes.	git add . git add src/ git add -p (patch/hunk mode)
git commit	Create commit from staged changes.	git commit -m 'feat: add payment service'
git push	Upload commits to remote.	git push origin feature/payment
git pull	Fetch + merge from remote.	git pull --rebase origin main # rebase instead of merge
git fetch	Download remote changes without merging.	git fetch origin # then review before merge
git log	Show commit history.	git log --oneline --graph --all # visual tree
git diff	Show unstaged changes.	git diff HEAD git diff main..feature

git stash	Temporarily save uncommitted work.	git stash push -m 'wip: fixing auth' git stash pop
------------------	------------------------------------	--

2.2 Branching & Merging

Command	What It Does
git branch feature/login	Create new branch.
git checkout -b feature/login	Create and switch to new branch.
git switch -c feature/login	Modern syntax to create and switch (Git 2.23+).
git merge feature/login	Merge branch into current. Creates merge commit if diverged.
git merge --squash feature/login	Squash all commits into one staged change — then commit manually.
git merge --no-ff feature/login	Force merge commit even if fast-forward possible. Preserves branch history.
git rebase main	Replay current branch commits on top of main. Linear history.
git rebase -i HEAD~3	Interactive rebase — squash, reorder, edit, drop last 3 commits.
git cherry-pick abc1234	Apply specific commit(s) onto current branch.
git branch -d feature/login	Delete branch (safe — warns if unmerged).

2.3 Undoing Changes

Command	What It Does	Safe?
git restore file.txt	Discard unstaged changes in working dir.	Yes — local only
git restore --staged file.txt	Unstage file (keep working dir changes).	Yes
git commit --amend	Modify last commit (message or content). Rewrites history.	Only if not pushed
git reset --soft HEAD~1	Undo last commit. Keep changes staged.	Only if not pushed
git reset --mixed HEAD~1	Undo last commit. Keep changes unstaged (default).	Only if not pushed
git reset --hard HEAD~1	Undo last commit AND discard changes. Destructive.	Never on shared branches
git revert HEAD	Create new commit that undoes HEAD. Safe for shared branches.	Yes — doesn't rewrite history
git reflog	History of HEAD movements. Recover 'lost' commits after reset.	Read-only

3. Branching Strategies

3.1 Git Flow

- **main**: production-ready code only. Tagged releases.
 - **develop**: integration branch. All features merged here first.
 - **feature/xxx**: branched from develop. Merged back to develop via PR.
 - **release/x.y.z**: branched from develop for stabilisation. Merged to main + develop.
 - **hotfix/xxx**: branched from main for urgent prod fixes. Merged to main + develop.
- ✓ Good for: scheduled releases, versioned software, multiple versions in production.
- Drawback: complex, long-lived branches, integration hell. Not ideal for CI/CD.

3.2 GitHub Flow (Trunk-Based for Web Apps)

- **main**: always deployable. All CI/CD pipelines trigger from main.
- Create short-lived feature branch → open PR → review → merge to main → auto-deploy.
- No develop or release branches. Simplicity is the goal.
- ✓ Good for: web apps with continuous deployment. Teams deploying multiple times per day.

3.3 Trunk-Based Development

- Developers commit directly to main (trunk) or very short-lived branches (< 1 day).
- Feature flags control incomplete feature visibility in production.
- Requires strong CI — all tests pass before merge.
- ✓ Good for: mature teams, high deployment frequency, Google/Facebook-style DevOps.

3.4 GitLab Flow

- Adds environment branches to GitHub Flow: main → staging → production.
- Merge to main → auto-deploy to staging. Merge to production → deploy to prod.
- ✓ Good for: teams needing staging environments between code and production.

4. Advanced Git Concepts

4.1 Merge vs Rebase

	Merge	Rebase
History	Preserves true history — shows exact timeline of when branches diverged.	Creates linear history — rewrites commits as if always on top of target.
Commit graph	Creates merge commit with two parents.	New commits with new SHAs. Original commits untouched.
Conflicts	Resolved once at merge time.	May resolve same conflict for each commit being replayed.
Safe on shared?	Yes — never rewrites history.	Never rebase shared/public branches — rewrites history breaks others.
Use when	Merging completed feature branches, preserving feature history.	Keeping feature branch updated with main. Cleaning up local commits before PR.

4.2 Useful Advanced Commands

```
# Interactive rebase – clean up commits before PR
git rebase -i HEAD~4 # squash 4 commits into one

# Find commit that introduced a bug
git bisect start
git bisect bad # current commit is broken
git bisect good v1.2.0 # last known good tag
# Git checks out midpoint – mark good/bad until found

# Show who changed each line
git blame src/OrderService.java

# Search commit messages
git log --grep='payment' --oneline

# Show changes introduced by a commit
```

```
git show abc1234
# Partial staging – stage only specific hunks
git add -p OrderService.java
# Recover 'lost' commit after bad reset
git reflog # find SHA
git checkout -b recovery abc1234
```

5. Commit Conventions

5.1 Conventional Commits

- Format: **type(scope): description**

Type	When to Use	Example
feat	New feature for the user.	feat(payment): add Stripe payment gateway
fix	Bug fix.	fix(auth): resolve token expiry race condition
refactor	Code restructure, no feature/fix change.	refactor(order): extract discount calculation
perf	Performance improvement.	perf(query): add composite index on orders(user_id, status)
test	Adding or fixing tests.	test(payment): add integration tests for refund flow
docs	Documentation only.	docs(api): update OpenAPI spec for /orders endpoint
chore	Maintenance, dependency updates, build.	chore(deps): upgrade Spring Boot to 3.2.0
ci	CI/CD pipeline changes.	ci: add SonarQube quality gate to pipeline
BREAKING CHANGE	In footer or ! after type — incompatible API change.	feat!: remove deprecated /v1/orders endpoint

6. Bitbucket

6.1 Bitbucket vs GitHub vs GitLab

Feature	Bitbucket	GitHub	GitLab
Owned by	Atlassian	Microsoft	GitLab Inc.
CI/CD	Bitbucket Pipelines (bitbucket-pipelines.yml)	GitHub Actions (.github/workflows/)	GitLab CI (.gitlab-ci.yml) — most mature
Code review	Pull Requests. Inline comments. Diff view.	Pull Requests. Reviews. GitHub Copilot.	Merge Requests. Approval rules. Code owners.
Atlassian integration	Native — Jira, Confluence, Trello, Bamboo.	Via marketplace	Via integrations
Branch permissions	Branch restrictions, merge checks, required builds.	Branch protection rules, CODEOWNERS.	Protected branches, approval rules, code owners.
Self-hosted	Bitbucket Data Center/Server	GitHub Enterprise	GitLab CE (free) / EE (paid)
Best for	Atlassian shops, Jira-integrated teams.	Open source, public repos, wide ecosystem.	Full DevSecOps platform, strong CI/CD.

6.2 Bitbucket Pipelines

```
# bitbucket-pipelines.yml – Spring Boot CI/CD
```

```

image: maven:3.9-eclipse-temurin-21
pipelines:
pull-requests:
  '**':
    - step:
      name: Test & Build
      caches: [maven]
      script:
        - mvn test
        - mvn package -DskipTests
branches:
main:
  - step:
      name: Build & Push Image
      script:
        - docker build -t myapp:$BITBUCKET_COMMIT .
        - docker push myregistry/myapp:$BITBUCKET_COMMIT
      - step:
          name: Deploy to Production
          deployment: production
          script:
            - kubectl set image deploy/myapp app=myregistry/myapp:$BITBUCKET_COMMIT

```

7. Interview Questions

Git Core

Q1. What is the difference between git merge and git rebase?

→ Merge: combines two branches, creates merge commit, preserves true history. Rebase: replays commits on top of target branch, creates linear history, rewrites commit SHAs. Use merge for completed features (preserves history). Use rebase to update feature branch with main or clean up local commits before PR. Never rebase shared/public branches.

Q2. What is git cherry-pick and when would you use it?

→ Applies one or more specific commits from one branch onto another. Use when: hotfix committed to wrong branch (cherry-pick to main), selectively pulling one feature from a branch that has mixed commits, backporting a fix to an older release branch. `git cherry-pick abc1234` creates new commit with same changes but new SHA.

Q3. How would you recover from a bad git reset --hard?

→ `git reflog` shows history of all HEAD movements including resets. Find the SHA of the commit before the reset. `git checkout -b recovery-branch SHA` to restore. `reflog` entries expire after 90 days by default. Always use `git revert` instead of `reset` on shared branches.

Q4. What is git bisect and when is it useful?

→ Binary search through commit history to find which commit introduced a bug. `git bisect start`, mark current as bad, mark known-good commit, Git checks out midpoint — you test and mark good/bad. Repeats until offending commit found. Useful for: regressions found weeks later with hundreds of commits in between.

Workflows

Q5. What branching strategy would you recommend for a team doing CI/CD?

→ Trunk-based development or GitHub Flow. Short-lived feature branches (under 1 day ideally), merge to main frequently. Feature flags for incomplete work. Strong automated test suite required. Avoids long-lived branch integration hell. Git Flow is better for scheduled, versioned releases but adds complexity for frequent deployments.

Q6. What is a merge conflict and how do you resolve it?

→ Occurs when two branches modify the same lines of a file differently. Resolution: `git status` shows conflicted files. Open file — conflict markers show `<<< HEAD (yours) === (divider) >>> branch (theirs)`. Edit to desired state, remove markers. `git add` resolved files. `git commit (merge)` or `git rebase --continue (rebase)`. Use `git mergetool` for visual 3-way merge.

Q7. What are Bitbucket branch restrictions?

→ Bitbucket lets you protect branches: require minimum approvals before merge, require successful builds (CI), prevent deletion, restrict who can push directly. PR checks: required reviewers, passing pipelines, no unresolved comments, minimum approvals. Use to enforce code review on main/develop.